

Certified path tracking and applications

Alexandre Guillemot

September 9, 2026 | Inria Saclay



In this thesis, I design and implement algorithms that manipulate mathematical objects.

$$\text{Solve } -13x^2 - 3x + 1 = 0.$$

Numerical analysis

$$\begin{cases} r_1 \approx -0.415 \\ r_2 \approx 0.185 \end{cases}$$

- Approximate representation
- Numerical computations

In this thesis, I design and implement algorithms that manipulate mathematical objects.

$$\text{Solve } -13x^2 - 3x + 1 = 0.$$

Numerical analysis

$$\begin{cases} r_1 \approx -0.4157788 \dots \\ r_2 \approx 0.1850096 \dots \end{cases}$$

- Approximate representation
- Numerical computations

In this thesis, I design and implement algorithms that manipulate mathematical objects.

$$\text{Solve } -13x^2 - 3x + 1 = 0.$$

Numerical analysis

$$\begin{cases} r_1 \approx -0.4157788 \dots \\ r_2 \approx 0.1850096 \dots \end{cases}$$

- Approximate representation
- Numerical computations

Computer algebra

$$\begin{cases} r_1 = \frac{-3+\sqrt{61}}{26} \\ r_2 = \frac{-3-\sqrt{61}}{26} \end{cases}$$

- Exact representation
- Symbolic computations

In this thesis, I design and implement algorithms that manipulate mathematical objects.

$$\text{Solve } -13x^2 - 3x + 1 = 0.$$

Numerical analysis

$$\begin{cases} r_1 \approx -0.4157788 \dots \\ r_2 \approx 0.1850096 \dots \end{cases}$$

- Approximate representation
- Numerical computations

Symbolic-numeric

$$\begin{cases} r_1 \in [-0.415 \pm 0.001] \\ r_2 \in [0.185 \pm 0.001] \end{cases}$$

- Exact input
- Numerical computations
+ certificates error bounds
- Recover exact information

Computer algebra

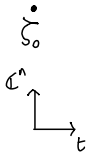
$$\begin{cases} r_1 = \frac{-3+\sqrt{61}}{26} \\ r_2 = \frac{-3-\sqrt{61}}{26} \end{cases}$$

- Exact representation
- Symbolic computations

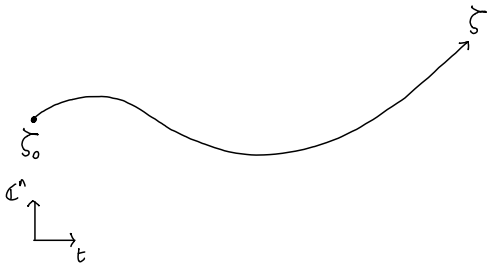
$$F_0(\zeta_0) = 0.$$

Input

- $F : \mathbb{C} \times \mathbb{C}^n \rightarrow \mathbb{C}^n$;
parametrized polynomial system
- $\zeta_0 \in \mathbb{C}^n$ zero of F_0 .



$$F_t(\zeta_t) = 0, \forall t \in [0, 1].$$

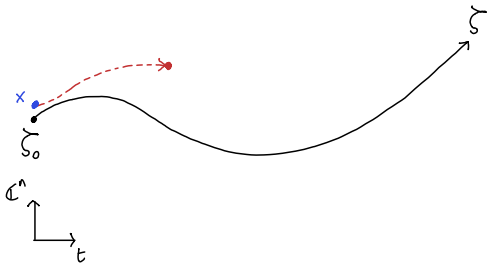


Input

- $F : \mathbb{C} \times \mathbb{C}^n \rightarrow \mathbb{C}^n$;
parametrized polynomial system
- $\zeta_0 \in \mathbb{C}^n$ zero of F_0 .

Goal. Compute ζ .

$$F_t(\zeta_t) = 0, \forall t \in [0, 1].$$



Input

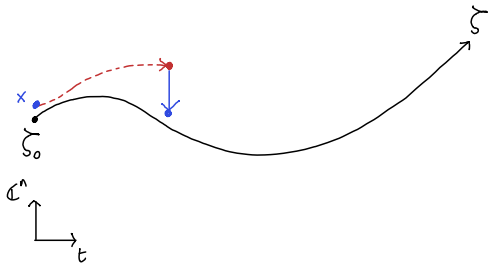
- $F : \mathbb{C} \times \mathbb{C}^n \rightarrow \mathbb{C}^n$;
parametrized polynomial system
- $\zeta_0 \in \mathbb{C}^n$ zero of F_0 .

Goal. Compute ζ .

def Track(F, x):

```
1  $t \leftarrow 0$ 
2 while  $t < 1$ :
3    $t, x \leftarrow \text{Predict}(F, t, x)$ 
4    $x \leftarrow \text{Correct}(F, t, x)$ 
```

$$F_t(\zeta_t) = 0, \forall t \in [0, 1].$$



Input

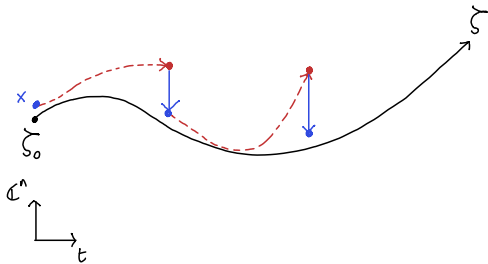
- $F : \mathbb{C} \times \mathbb{C}^n \rightarrow \mathbb{C}^n$;
parametrized polynomial system
- $\zeta_0 \in \mathbb{C}^n$ zero of F_0 .

Goal. Compute ζ .

def Track(F, x):

```
1   $t \leftarrow 0$ 
2  while  $t < 1$ :
3       $t, x \leftarrow \text{Predict}(F, t, x)$ 
4       $x \leftarrow \text{Correct}(F, t, x)$ 
```

$$F_t(\zeta_t) = 0, \forall t \in [0, 1].$$



Input

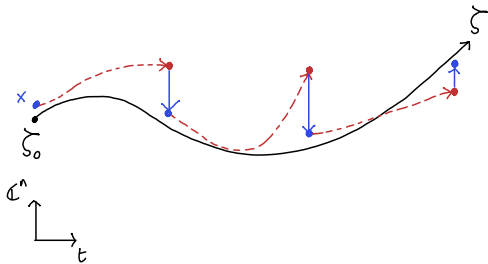
- $F : \mathbb{C} \times \mathbb{C}^n \rightarrow \mathbb{C}^n$;
parametrized polynomial system
- $\zeta_0 \in \mathbb{C}^n$ zero of F_0 .

Goal. Compute ζ .

def Track(F, x):

```
1   $t \leftarrow 0$ 
2  while  $t < 1$ :
3       $t, x \leftarrow \text{Predict}(F, t, x)$ 
4       $x \leftarrow \text{Correct}(F, t, x)$ 
```

$$F_t(\zeta_t) = 0, \forall t \in [0, 1].$$



Input

- $F : \mathbb{C} \times \mathbb{C}^n \rightarrow \mathbb{C}^n$;
parametrized polynomial system
- $\zeta_0 \in \mathbb{C}^n$ zero of F_0 .

Goal. Compute ζ .

def Track(F, x):

```
1  $t \leftarrow 0$ 
2 while  $t < 1$ :
3    $t, x \leftarrow \text{Predict}(F, t, x)$ 
4    $x \leftarrow \text{Correct}(F, t, x)$ 
```

$$F_t(x) = (11i + 2)x^5t^2 + 8ix^5t + 5xt + (-i + 2)x - t - 2i.$$

t variable

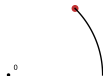
0

Roots of F_t

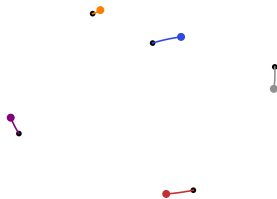


$$F_t(x) = (11i + 2)x^5t^2 + 8ix^5t + 5xt + (-i + 2)x - t - 2i.$$

t variable

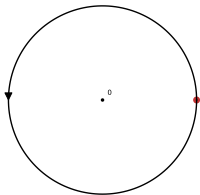


Roots of F_t

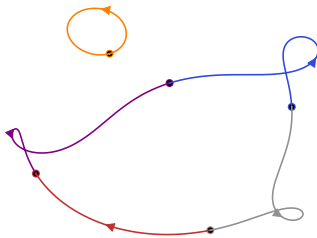


$$F_t(x) = (11i + 2)x^5t^2 + 8ix^5t + 5xt + (-i + 2)x - t - 2i.$$

t variable



Roots of F_t



$$F_t(x) = (11i + 2)x^5t^2 + 8ix^5t + 5xt + (-i + 2)x - t - 2i.$$

Monodromy permutation

Permutation of the roots associated to a parameter loop.

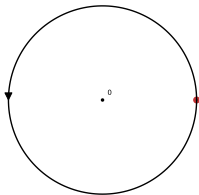
! discriminant

Monodromy group

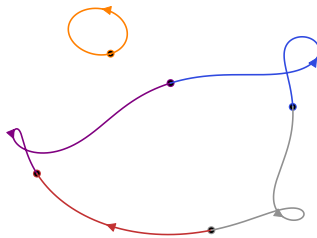
Subgroup of S_5 generated by all monodromy permutations.

⚙ here, two loops suffice

t variable

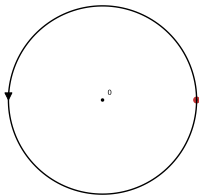


Roots of F_t

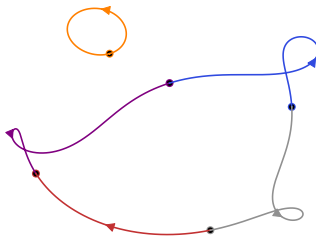


$$F_t(x) = (11i + 2)x^5t^2 + 8ix^5t + 5xt + (-i + 2)x - t - 2i.$$

t variable



Roots of F_t



Monodromy permutation

Permutation of the roots associated to a parameter loop.

! discriminant

Monodromy group

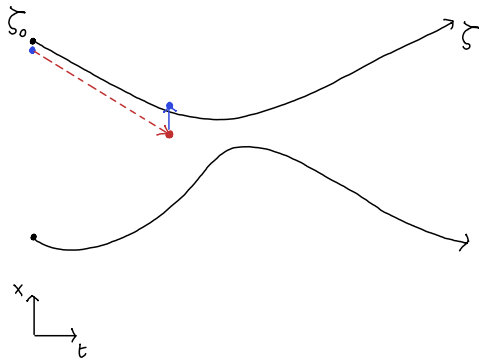
Subgroup of S_5 generated by all monodromy permutations.

⚙ here, two loops suffice

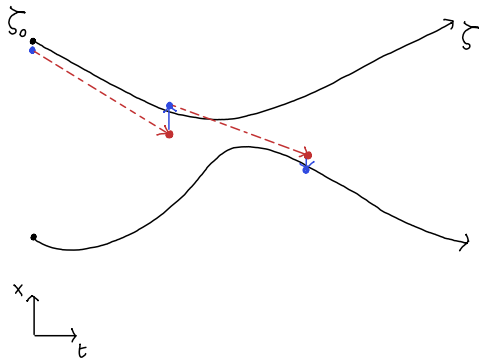
Theorem (Hermite, 1851)

Monodromy group $\simeq$ Galois group of $F \in \mathbb{C}(t)[x]$.

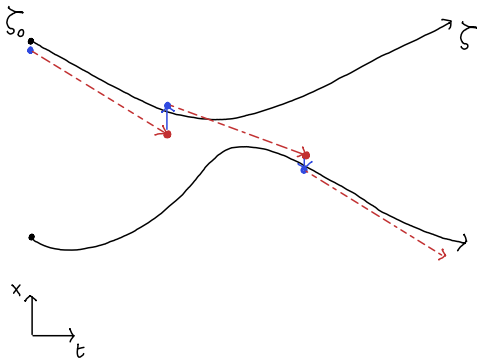
💡 Compute the root paths using homotopy continuation.



Path jumping: the algorithm switches paths, leading to a silent error.

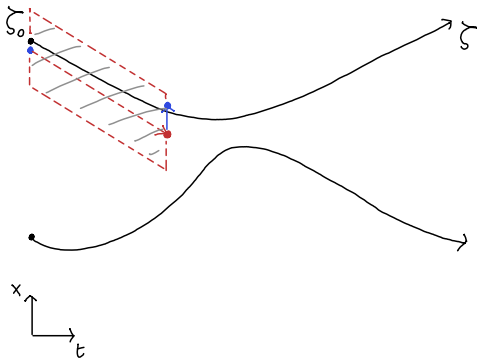


Path jumping: the algorithm switches paths, leading to a silent error.



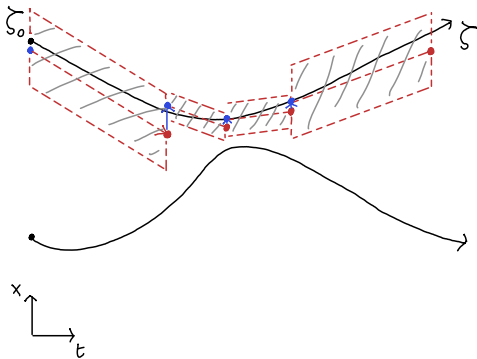
Path jumping: the algorithm switches paths, leading to a silent error.

For monodromy, we cannot tolerate path jumping.



Path jumping: the algorithm switches paths, leading to a silent error.

For monodromy, we cannot tolerate path jumping.

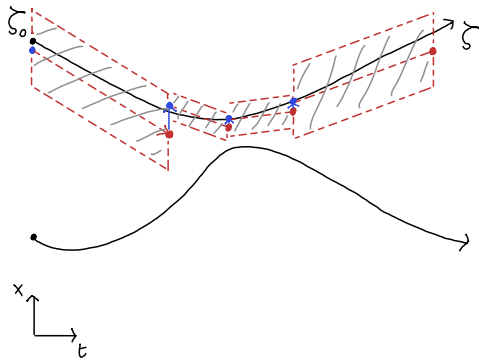


Path jumping: the algorithm switches paths, leading to a silent error.

For monodromy, we cannot tolerate path jumping.

Solution path certification

- + Correct path is tracked.
- Computational cost.



Path jumping: the algorithm switches paths, leading to a silent error.

For monodromy, we cannot tolerate path jumping.

Solution path certification

- + Correct path is tracked.
- Computational cost.

Motivation

Enumerative geometry, algebraic topology over complex varieties.

monodromy/braid computations

Faster certified homotopy continuation

- Validated Numerics for Algebraic Path Tracking. (2024). *Proc. ISSAC 2024*, 36–45. With P. Lairez.
- Certified Algebraic Path Tracking with Alpath. (2026). *ACM Commun. Comput. Algebra*, 59(3), 53–56.

Application 1. Belyi maps

- Belyi map verification using certified path tracking. (2026). To appear in ICMS proceedings. With J. Voight.

Application 2. Braid computations

- *Computing braids from approximate data*. (2026). arXiv: 2601.23073 [cs]. With P. Lairez.

Certified homotopy continuation

Belyi map monodromy

Braid computations

Core idea: Drexler, 1977; Garcia and Zangwill, 1979.

Software:

- HOM4PS family (Chen, Lee, & Li, 2014);
- PHCpack (Verschelde, 1999);
- PHoM (Gunji, Kim, Kojima, Takeda, Fujisawa, & Mizutani, 2004);
- POLSYS_GLP (Su, McCarthy, Sosonkina, & Watson, 2006);
- Macaulay2, NumericalAlgebraicGeometry package (Leykin, 2011);
- Bertini (Bates, Sommese, Hauenstein, & Wampler, 2013);
- HomotopyContinuation.jl (Breiding & Timme, 2018).

Robust homotopy continuation: Telen, Van Barel, and Verschelde, 2020.

Endpoints certification: Rump, 1983; Smale, 1986; Hauenstein and Sottile, 2012; Breiding, Rose, and Timme, 2023.

Complexity analysis: Smale, 1986; Shub and Smale, 1993; Shub and Smale, 1994.

Explicit stepsize bounds: Beltrán, 2011; Dedieu, Malajovich, and Shub, 2013; Bürgisser and Cucker, 2011.

Algorithms and implementations:

- Macaulay2, NumericalAlgebraicGeometry package (Beltrán & Leykin, 2012); BSS model
- Beltrán and Leykin, 2013; Turing machine model
- Hauenstein, Haywood, and Liddell, 2014; Newton homotopies, a posteriori certification
- Hauenstein and Liddell, 2016. Newton homotopies

Interval Gauss-Seidel: Kearfott and Xing, 1994.

Krawczyk's method:

- van der Hoeven, 2015; Taylor models
- Duff and Lee, 2024. developed in parallel to the work of this thesis

Stepsize bounds:

- Schneps, 1994;
- Kranich, 2016;
- Bruin, Sijssling, and Zotine, 2019.

Compute-and-connect fibers:

- Deconinck and van Hoeij, 2001;
- Bartholdi, Buff, Graf Von Bothmer, and Kröker, 2015.

Interval Newton: SIROCCO (Marco-Buzunariz & Rodríguez, 2016); available in Sage

Root counting and subdivision: Xu, Burr, and Yap, 2018.

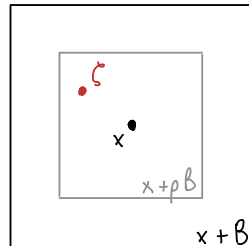
Krawczyk, 1969; Moore, 1977; Rump, 1983

- $f : \mathbb{C}^n \rightarrow \mathbb{C}^n$ polynomial, $\rho \in (0, 1)$,
- $x \in \mathbb{C}^n$, $A \in \mathbb{C}^{n \times n}$, $B \subseteq \mathbb{C}^n$ a ball of center 0,

such that for all $u, v \in B$,

$$-A \cdot f(x) + [I_n - A \cdot df(x + u)]v \in \rho B.$$

Then f has a zero in $x + \rho B$, unique in $x + B$.



• other zero of f

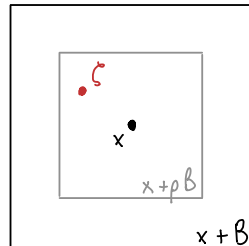
Krawczyk, 1969; Moore, 1977; Rump, 1983

- $f : \mathbb{C}^n \rightarrow \mathbb{C}^n$ polynomial, $\rho \in (0, 1)$,
- $x \in \mathbb{C}^n$, $A \in \mathbb{C}^{n \times n}$, $B \subseteq \mathbb{C}^n$ a ball of center 0,

such that for all $u, v \in B$,

$$-A \cdot f(x) + [I_n - A \cdot df(x + u)]v \in \rho B.$$

Then f has a zero in $x + \rho B$, unique in $x + B$.



• other zero of f

Proof sketch

We show that $\varphi : x + B \rightarrow \mathbb{C}^n$ defined by $\varphi(w) = w - Af(w)$ is a ρ -contraction map with values in $x + \rho B$.

Definition

A ρ -Moore box for f is a triple (x, B, A) such that $-Af(x) + [I_n - Adf(x + B)]B \subseteq \rho B$.

Problem

How do we **effectively** check that for all $u, v \in B$,

$$-Af(x) + [I_n - Adf(x + u)]v \in \rho B?$$

Interval arithmetic

- Define interval binary operations $\boxplus$ and $\boxtimes$. Key property:

$$\text{for all } x \in I, y \in J, \quad x + y \in I \boxplus J \text{ and } xy \in I \boxtimes J.$$

- f polynomial function. Can extend it to intervals using $\boxplus$ and $\boxtimes$. Key property:

$$\text{for all } x \in I, \quad f(x) \in \square f(I).$$

- **Krawczyk's operator** is a polynomial in u, v .

Exact model

- Interval endpoints: $\mathbb{Q}$;
- $[a, b] \boxplus [c, d] = [a + c, b + d]$;
- $[a, b] \boxtimes [c, d] = [\min\{ac, ad, bc, bd\}, \max\{ac, ad, bc, bd\}]$.

Pros and cons

- ✓ Good theoretical properties.
- ✗ Coefficient swell.

Double precision model

- Interval endpoints: {IEEE-754 64-bits floating-point numbers};
- $[a, b] \boxplus [c, d] = [\underline{a + c}, \overline{b + d}]$;
- $[a, b] \boxtimes [c, d] = [\min\{\underline{ac}, \underline{ad}, \underline{bc}, \underline{bd}\}, \max\{\overline{ac}, \overline{ad}, \overline{bc}, \overline{bd}\}]$.

Pros and cons

- ✓ Fast.
- ✗ Bad theoretical properties.
- ✗ Not enough representable numbers.

MPFI model

- Interval endpoints: $\{m2^e \in \mathbb{R} : m, e \in \mathbb{Z}\}$;
- precision $P \in \mathbb{Z}_{>}$ (number of bits) that can be changed at will;
- interval operations: endpoints rounding done at precision P .

Pros and cons

- ✓ Good theoretical properties as $P \rightarrow \infty$.
- ✓ Fast when we can maintain low precision.
- ⚙ Need to carefully manage P in the algorithms to ensure termination.

- ✓ Isolate a zero of a polynomial function.

Moore boxes

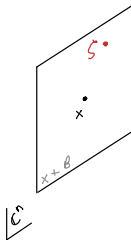
- ⚙️ Isolate a solution path.

Moore's criterion

$x \in \mathbb{C}^n$, B centered ball, $A \in \mathbb{C}^{n \times n}$. If for all $u, v \in B$

$$-Af(x) + [I_n - Adf(x + u)]v \in \rho B,$$

then $x + B$ isolates a zero of f .



- ✓ Isolate a zero of a polynomial function.

Moore boxes

- ⚙️ Isolate a solution path.

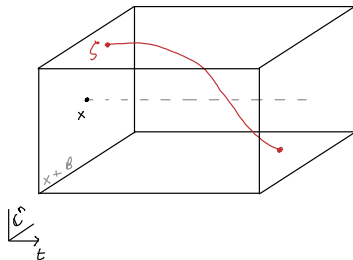
Moore's criterion on an interval

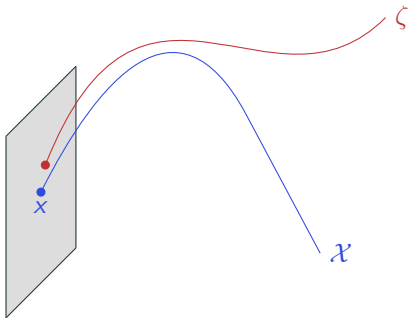
$x \in \mathbb{C}^n$, B centered ball, $A \in \mathbb{C}^{n \times n}$. If for all $u, v \in B$ and $t \in I$

$$-Af_t(x) + [I_n - Adf_t(x+u)]v \in \rho B,$$

then $x + B$ isolates a zero of f_t , for all $t \in I$.

- 💡 Again, replace t by I and use interval arithmetic.

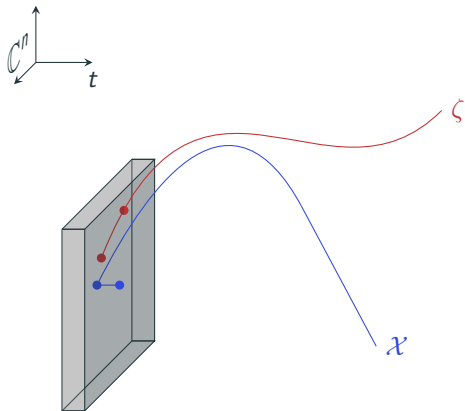




Predictor

A map $\mathcal{X} : \mathbb{R} \rightarrow \mathbb{C}^n$.

In practice, one should have $\mathcal{X}(s) \approx \zeta_{t+s}$ around 0.



Predictor

A map $\mathcal{X} : \mathbb{R} \rightarrow \mathbb{C}^n$.

In practice, one should have $\mathcal{X}(s) \approx \zeta_{t+s}$ around 0.

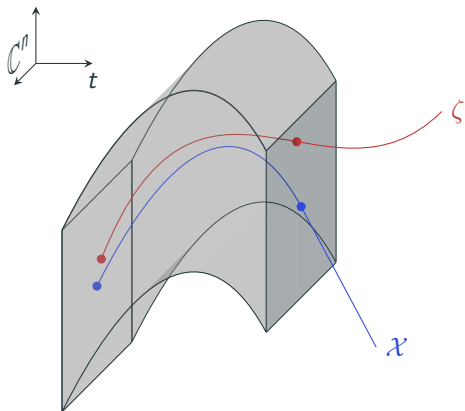
How to extend?

Pb: check that for all $s \in [0, \delta]$, (x, B, A) is a ρ -Moore box for F_{t+s} .

Soln: try

$$-AF_{t+\textcolor{red}{s}}(x) + [1 - \text{Ad}F_{t+\textcolor{red}{s}}(x + B)]B \subseteq \rho B,$$

where $\textcolor{red}{S} = [0, \delta]$.



Predictor

A map $\mathcal{X} : \mathbb{R} \rightarrow \mathbb{C}^n$.

In practice, one should have $\mathcal{X}(s) \approx \zeta_{t+s}$ around 0.

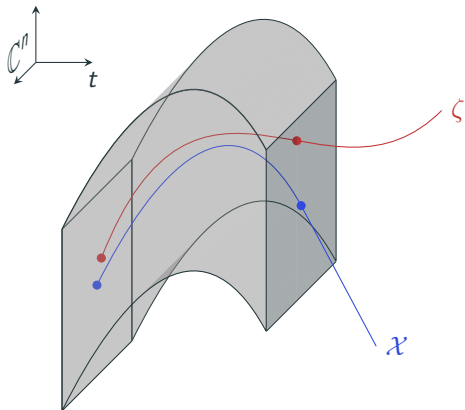
How to extend **along** $\mathcal{X}$?

Pb: check that for all $s \in [0, \delta]$, $(\mathcal{X}(s), B, A)$ is a ρ -Moore box for F_{t+s} .

Soln? try

$$-AF_{t+s}(\mathcal{X}(S)) + [1 - AdF_{t+s}(\mathcal{X}(S) + B)]B \subseteq \rho B,$$

where $S = [0, \delta]$.



Predictor

A map $\mathcal{X} : \mathbb{R} \rightarrow \mathbb{C}^n$.

In practice, one should have $\mathcal{X}(s) \approx \zeta_{t+s}$ around 0.

How to extend **along** $\mathcal{X}$?

Pb: check that for all $s \in [0, \delta]$, $(\mathcal{X}(s), B, A)$ is a ρ -Moore box for F_{t+s} .

Soln? try

$$-AF_{t+s}(\mathcal{X}(S)) + [1 - AdF_{t+s}(\mathcal{X}(S) + B)]B \subseteq \rho B,$$

where $S = [0, \delta]$.

This is too strong!

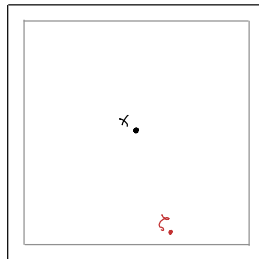
Way around the dependency problem: **Taylor models**.

- ✓ Check that a Moore-box isolates the solution path on an interval.
- ⚙ How do we choose the candidate Moore-box and the interval?
- ⚙ Correction step?

- ✓ Check that a Moore-box isolates the solution path on an interval.
- ⚙ How do we choose the candidate Moore-box and the interval?
- ⚙ Correction step?

```
def Refine( $f, x, B, A$ ):
```

Input. (x, B, A) a $\frac{7}{8}$ -Moore box for f .



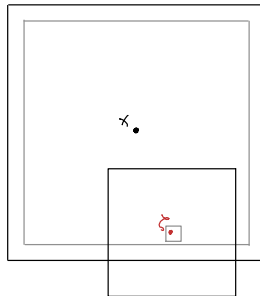
other zero

- ✓ Check that a Moore-box isolates the solution path on an interval.
- ⚙ How do we choose the candidate Moore-box and the interval?
- ⚙ Correction step?

def Refine(f, x, B, A):

Input. (x, B, A) a $\frac{7}{8}$ -Moore box for f .

Output. a $\frac{1}{8}$ -Moore box for f . isolating the same zero



✓ Check that a Moore-box isolates the solution path on an interval.

⚙️ How do we choose the candidate Moore-box and the interval?

⚙️ Correction step?

def Refine(f, x, B, A):

Input. (x, B, A) a $\frac{7}{8}$ -Moore box for f .

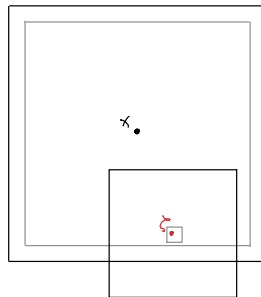
Output. a $\frac{1}{8}$ -Moore box for f . isolating the same zero

Idea. Goal:

$$-Af(x) + [I_n - Adf(x + B)]B \subseteq \frac{1}{8}B.$$

If **left term** small, shrink B .

If **left term** large, do a quasi-Newton iteration.



other zero

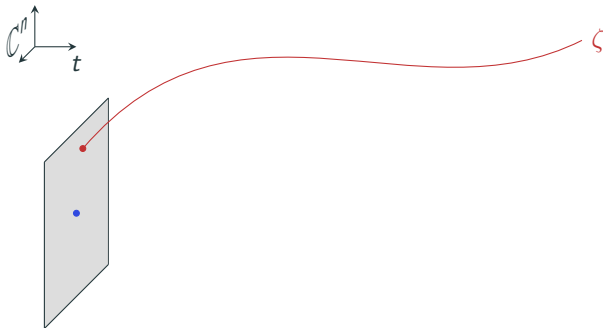
- ✓ Check that a Moore-box isolates the solution path on an interval.
- ⚙️ How do we choose the candidate Moore-box and the interval?
- ✓ Correction step: Refine. transforms a $\frac{7}{8}$ -Mb into a $\frac{1}{8}$ -Mb

def Track(F, m):

Input. m a Moore box for F_0 .

```

1   $0 \leftarrow t, \delta \leftarrow 1$ 
2  while  $t < 1$ :
3       $m \leftarrow \text{Refine}(F_t, m)$ 
4      try to extend  $m$  to  $[t, t + \delta]$ 
5      if it fails, halve  $\delta$  and retry Line 4
6       $t \leftarrow t + \delta$ 
    
```



- ✓ Check that a Moore-box isolates the solution path on an interval.
- ⚙️ How do we choose the candidate Moore-box and the interval?
- ✓ Correction step: Refine. transforms a $\frac{7}{8}$ -Mb into a $\frac{1}{8}$ -Mb

def Track(F, m):

Input. m a Moore box for F_0 .

1 $0 \leftarrow t, \delta \leftarrow 1$

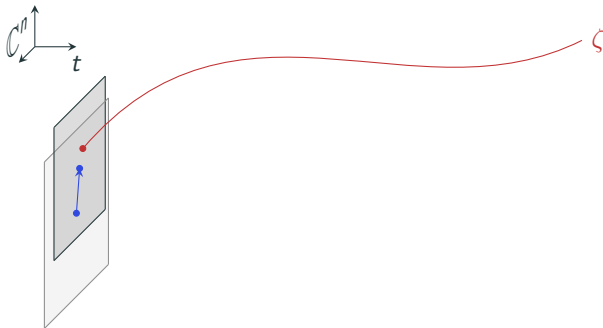
2 **while** $t < 1$:

3 $m \leftarrow \text{Refine}(F_t, m)$

4 try to extend m to $[t, t + \delta]$

5 if it fails, halve δ and retry Line 4

6 $t \leftarrow t + \delta$



- ✓ Check that a Moore-box isolates the solution path on an interval.
- ⚙️ How do we choose the candidate Moore-box and the interval?
- ✓ Correction step: Refine. transforms a $\frac{7}{8}$ -Mb into a $\frac{1}{8}$ -Mb

def Track(F, m):

Input. m a Moore box for F_0 .

1 $0 \leftarrow t, \delta \leftarrow 1$

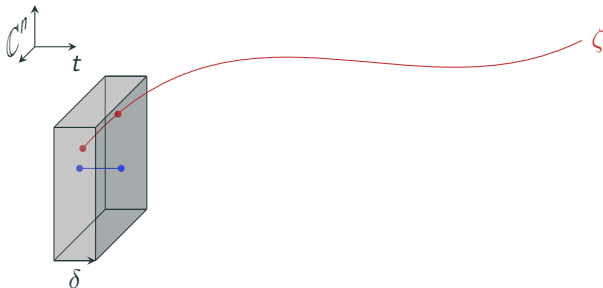
2 **while** $t < 1$:

3 $m \leftarrow \text{Refine}(F_t, m)$

4 try to extend m to $[t, t + \delta]$

5 if it fails, halve δ and retry Line 4

6 $t \leftarrow t + \delta$



- ✓ Check that a Moore-box isolates the solution path on an interval.
- ⚙️ How do we choose the candidate Moore-box and the interval?
- ✓ Correction step: Refine. transforms a $\frac{7}{8}$ -Mb into a $\frac{1}{8}$ -Mb

def Track(F, m):

Input. m a Moore box for F_0 .

1 $0 \leftarrow t, \delta \leftarrow 1$

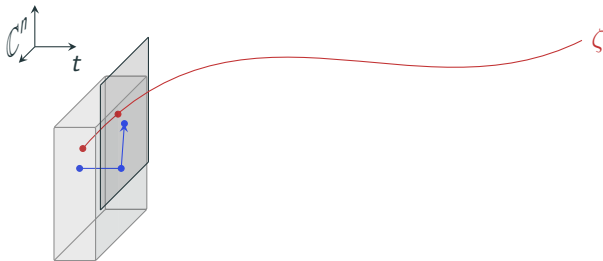
2 **while** $t < 1$:

3 $m \leftarrow \text{Refine}(F_t, m)$

4 try to extend m to $[t, t + \delta]$

5 if it fails, halve δ and retry Line 4

6 $t \leftarrow t + \delta$



✓ Check that a Moore-box isolates the solution path on an interval.

⚙️ How do we choose the candidate Moore-box and the interval?

✓ Correction step: Refine. transforms a $\frac{7}{8}$ -Mb into a $\frac{1}{8}$ -Mb

def Track(F, m):

Input. m a Moore box for F_0 .

1 $0 \leftarrow t, \delta \leftarrow 1$

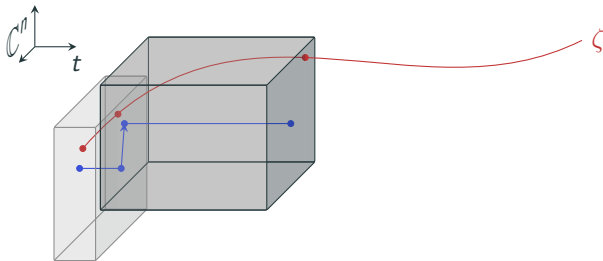
2 **while** $t < 1$:

3 $m \leftarrow \text{Refine}(F_t, m)$

4 try to extend m to $[t, t + \delta]$

5 if it fails, halve δ and retry Line 4

6 $t \leftarrow t + \delta$



- ✓ Check that a Moore-box isolates the solution path on an interval.
- ⚙️ How do we choose the candidate Moore-box and the interval?
- ✓ Correction step: Refine. transforms a $\frac{7}{8}$ -Mb into a $\frac{1}{8}$ -Mb

def Track(F, m):

Input. m a Moore box for F_0 .

1 $0 \leftarrow t, \delta \leftarrow 1$

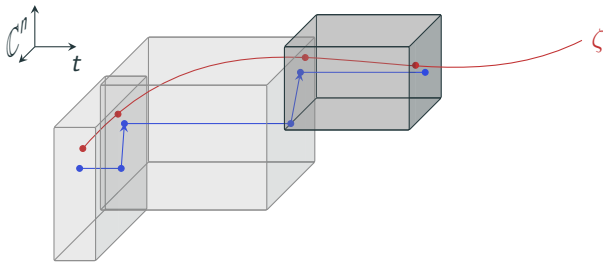
2 **while** $t < 1$:

3 $m \leftarrow \text{Refine}(F_t, m)$

4 try to extend m to $[t, t + \delta]$

5 if it fails, halve δ and retry Line 4

6 $t \leftarrow t + \delta$



- ✓ Check that a Moore-box isolates the solution path on an interval.
- ✓ How do we choose the candidate Moore-box and the interval?
- ✓ Correction step: Refine. transforms a $\frac{7}{8}$ -Mb into a $\frac{1}{8}$ -Mb

def Track(F, m):

Input. m a Moore box for F_0 .

1 $0 \leftarrow t, \delta \leftarrow 1$

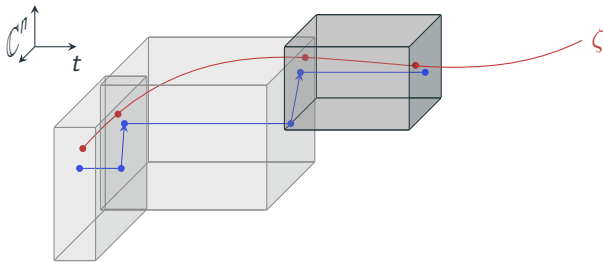
2 **while** $t < 1$:

3 $m \leftarrow \text{Refine}(F_t, m)$

4 try to extend m to $[t, t + \delta]$

5 if it fails, halve δ and retry Line 4

6 $t \leftarrow t + \delta$



Algpath: implementation of the presented algorithm.

- *Rust* implementation. ~10k lines of code
- Available at <https://gitlab.inria.fr/numag/algpath>.
- CLI, input described with a JSON file.
- SLP representation of the input.
- Parallelized over the input fiber.
- *Mixed precision*, that relies on a SIMD double precision IA and Arb¹.
- Automatic precision management.
- Rigorous computation of monodromy permutations.

¹Johansson, 2017.

name	dim.	deg.	HomotopyContinuation.jl			Algpath			Macaulay2	
			time (s)	jump.	steps	time (s)	prec.	steps	time (s)	steps
dense	1	10	1.4		10	< 0.1		31	0.1	2146
dense	1	100	1.7		51	0.5		794	> 3 h	
dense	1	1000	6.7		100	12 min	59	17 k	1000 fails	
katsura	11	2 ¹⁰	5.5		179	21		360	27 min	88 k
katsura	21	2 ²⁰	3 h		469	62 h	65	12 k	> 8 GB	
structured *	4	3 ⁴	3.5	✗	78	0.3		198	11	8925
structured *	8	3 ⁸	3.4		122	1.8		281	223	97 k
clustered	1	25	3.0	✗	151	0.3		837	179	677 k
clustered	1	50	3.0	✗	153	86	69	11 k	> 3 h	

Figure 1: Total degree homotopy benchmarks. A * means that only 100 random roots were tracked.

Certified homotopy continuation

Belyi map monodromy

Braid computations

A map $\phi : X \rightarrow \mathbb{P}^1$ unramified away from $\{0, 1, \infty\} \dots$

Special case: $X = \mathbb{P}^1$

A non-constant rational function $\frac{p}{q} \in \mathbb{C}(x)$. The roots of $\Delta_x(p - tq)$ are included in $\{0, 1\}$.

A map $\phi : X \rightarrow \mathbb{P}^1$ unramified away from $\{0, 1, \infty\}$...

Special case: $X = \mathbb{P}^1$

A non-constant rational function $\frac{p}{q} \in \mathbb{C}(x)$. The roots of $\Delta_x(p - tq)$ are included in $\{0, 1\}$.

Example (LMFDB entry 3T2-3.2.1.2.1-a, slightly modified)

- $p = 2x^3 - \frac{3}{2}x + \frac{1}{2}, \quad q = 1.$

- Belyi map?

```
sage: (p - t).discriminant(x).factor()  
(-108) * (t - 1) * t
```

Monodromy of the Belyi map: monodromy of $f_t(x) = p - tq$.

two loops suffice, around 0 and 1

Properties

- Belyi maps are uniquely determined by their monodromy for loops around 0 and 1 in $\mathbb{C}$.
- There exists a Belyi map from X if and only if X can be defined over $\mathbb{Q}^{\text{al}}$ (Belyi, 1980).
 $\rightsquigarrow \text{Gal}(\mathbb{Q}^{\text{al}} | \mathbb{Q})$ acts on Belyi maps

Properties

- Belyi maps are uniquely determined by their monodromy for loops around 0 and 1 in $\mathbb{C}$.
- There exists a Belyi map from X if and only if X can be defined over $\mathbb{Q}^{\text{al}}$ (Belyi, 1980).
 $\rightsquigarrow \text{Gal}(\mathbb{Q}^{\text{al}} | \mathbb{Q})$ acts on Belyi maps

Computational goal

Input. $\sigma_0, \sigma_1 \in S_d$. $\langle \sigma_0, \sigma_1 \rangle \subseteq S_d$ transitive subgroup

Output. Equations for the Belyi map ϕ with monodromy (σ_0, σ_1) .

See Sijsling and Voight, 2015; Roberts, 2018; Barth and Wenz, 2021.

Klug, Musty, Schiavone, and Voight, 2014 $\rightsquigarrow$ LMFDB Belyi map section.

Properties

- Belyi maps are uniquely determined by their monodromy for loops around 0 and 1 in $\mathbb{C}$.
- There exists a Belyi map from X if and only if X can be defined over $\mathbb{Q}^{\text{al}}$ (Belyi, 1980).
 $\rightsquigarrow \text{Gal}(\mathbb{Q}^{\text{al}} | \mathbb{Q})$ acts on Belyi maps

Computational goal

Input. $\sigma_0, \sigma_1 \in S_d$. $\langle \sigma_0, \sigma_1 \rangle \subseteq S_d$ transitive subgroup

Output. Equations for the Belyi map ϕ with monodromy (σ_0, σ_1) .

See Sijsling and Voight, 2015; Roberts, 2018; Barth and Wenz, 2021.

Klug, Musty, Schiavone, and Voight, 2014 $\rightsquigarrow$ LMFDB Belyi map section.

Verification step

All known methods require ultimately checking the monodromy of ϕ .

Using symbolic tools

- Elkies, 2013: compute and verify a Belyi map with monodromy group M_{23} .
- Barth and Wenz, 2021: compute and verify Belyi maps of large degree (55–280).

Homotopy continuation type certification

- Schneps, 1994; Bruin, Sijsling, and Zotine, 2019;
- Deconinck and van Hoeij, 2001; Bartholdi, Buff, Graf Von Bothmer, and Kröker, 2015.

Monodromy in general

- Hauenstein, Haywood, and Liddell, 2014;
- Duff and Lee, 2026.
- ...

Belyi map section of the LMFDB

- 1111 Belyi maps.
- Computed using the method of Klug, Musty, Schiavone, and Voight (2014).
- Contains Belyi maps up to degree 9, contains all Belyi maps with degree ≤ 6 .
- ! no rigorous monodromy verification.

Contribution

- Using Algpah, compute (rigorously) the monodromy of all 1111 Belyi maps entries.
- Along de way, develop Sage scripts to compute the monodromy of Belyi maps in general.
- Results and scripts available at <https://gitlab.inria.fr/aguillem/belyi-monodromy>.

Computation of all triples present in the LMFDB

- Whole computation: 17min on 6 threads, using an Intel Core i7-1370P CPU.
- 1.2h of CPU time.

Found bugs being handled

- Monodromy may only be correct up to an action of S_3 .
<https://github.com/LMFDB/lmfdb/issues/6796>
- Bug in the embeddings–monodromy correspondance for 7 entries.
<https://github.com/LMFDB/lmfdb/issues/6798>

Additionally...

Monodromy computations of the map provided by Barth and Wenz (2021).
The two largest maps (degree 266 and 280) are out of reach.

Certified homotopy continuation

Belyi map monodromy

Braid computations

Geometric braids

n points moving without colliding in $\mathbb{C}$
up to homotopy

**Combinatorial braids**

free group words in the letters $\sigma_1, \dots, \sigma_{n-1}$
up to explicit relations



Geometric braids

n points moving without colliding in $\mathbb{C}$
up to homotopy



Combinatorial braids

free group words in the letters $\sigma_1, \dots, \sigma_{n-1}$
up to explicit relations



Geometric braids

n points moving without colliding in $\mathbb{C}$
up to homotopy



Combinatorial braids

free group words in the letters $\sigma_1, \dots, \sigma_{n-1}$
up to explicit relations



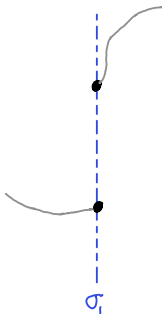
Geometric braids

n points moving without colliding in $\mathbb{C}$
up to homotopy

 $\leftrightarrow$

Combinatorial braids

free group words in the letters $\sigma_1, \dots, \sigma_{n-1}$
up to explicit relations

 $\mathbb{C}$

 σ_1

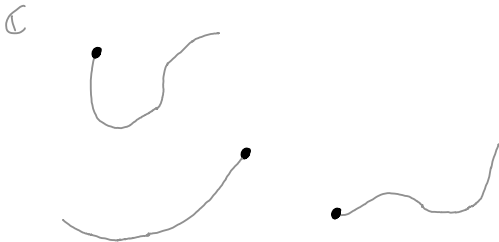
Geometric braids

n points moving without colliding in $\mathbb{C}$
up to homotopy

 $\leftrightarrow$

Combinatorial braids

free group words in the letters $\sigma_1, \dots, \sigma_{n-1}$
up to explicit relations


 σ_1

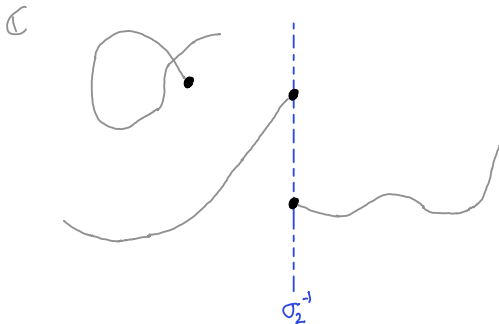
Geometric braids

n points moving without colliding in $\mathbb{C}$
up to homotopy

 $\leftrightarrow$

Combinatorial braids

free group words in the letters $\sigma_1, \dots, \sigma_{n-1}$
up to explicit relations



$$\sigma_1 \sigma_2^{-1}$$

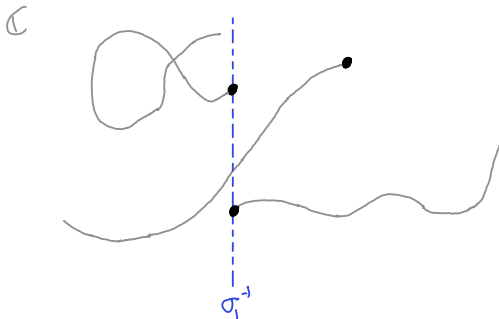
Geometric braids

n points moving without colliding in $\mathbb{C}$
up to homotopy

 $\leftrightarrow$

Combinatorial braids

free group words in the letters $\sigma_1, \dots, \sigma_{n-1}$
up to explicit relations



$$\sigma_1 \sigma_2^{-1} \sigma_1^{-1}$$

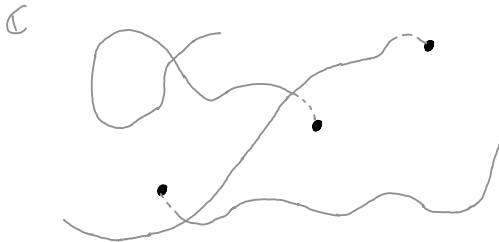
Geometric braids

n points moving without colliding in $\mathbb{C}$
up to homotopy



Combinatorial braids

free group words in the letters $\sigma_1, \dots, \sigma_{n-1}$
up to explicit relations



$$\sigma_1 \sigma_2^{-1} \sigma_1^{-1}$$

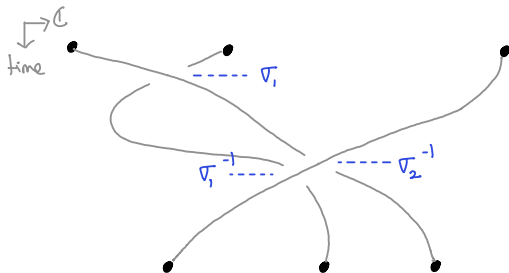
Geometric braids

n points moving without colliding in $\mathbb{C}$
up to homotopy

 $\leftrightarrow$

Combinatorial braids

free group words in the letters $\sigma_1, \dots, \sigma_{n-1}$
up to explicit relations



$$\sigma_1 \sigma_2^{-1} \sigma_1^{-1}$$

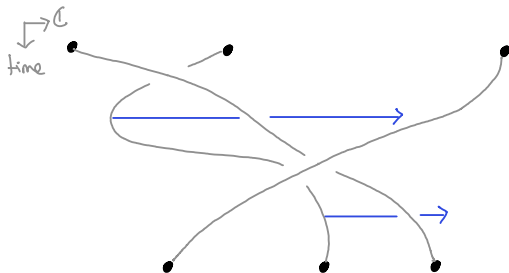
Geometric braids

n points moving without colliding in $\mathbb{C}$
up to homotopy

 $\Leftrightarrow$

Combinatorial braids

free group words in the letters $\sigma_1, \dots, \sigma_{n-1}$
up to explicit relations



$$\sigma_1 \sigma_2^{-1} \sigma_1^{-1}$$

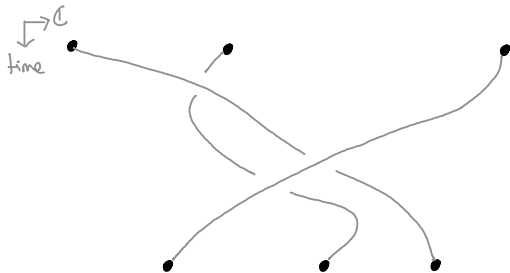
Geometric braids

n points moving without colliding in $\mathbb{C}$
up to homotopy

 $\Leftrightarrow$

Combinatorial braids

free group words in the letters $\sigma_1, \dots, \sigma_{n-1}$
up to explicit relations



$$\sigma_1 \sigma_2^{-1} \sigma_1^{-1}$$

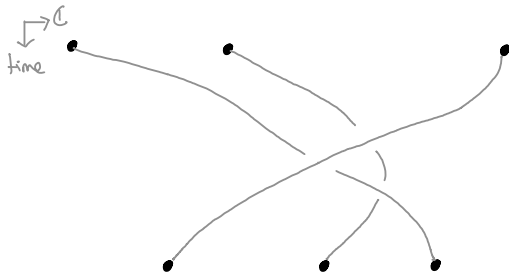
Geometric braids

n points moving without colliding in $\mathbb{C}$
up to homotopy

 $\leftrightarrow$

Combinatorial braids

free group words in the letters $\sigma_1, \dots, \sigma_{n-1}$
up to explicit relations



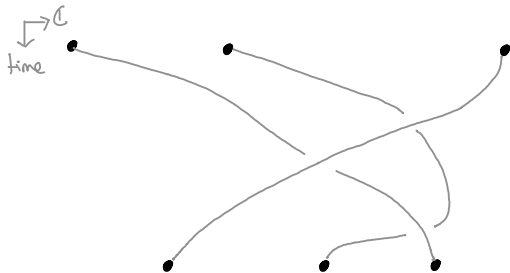
$$\sigma_1 \sigma_2^{-1} \sigma_1^{-1} = \sigma_2^{-1} \sigma_1^{-1} \sigma_2$$

Geometric braids

n points moving without colliding in $\mathbb{C}$
up to homotopy

 $\Leftrightarrow$ **Combinatorial braids**

free group words in the letters $\sigma_1, \dots, \sigma_{n-1}$
up to explicit relations



$$\sigma_1 \sigma_2^{-1} \sigma_1^{-1} = \sigma_2^{-1} \sigma_1^{-1} \sigma_2$$

Let $g \in \mathbb{C}[t][z]$. Moving t continuously in $\mathbb{C}$ induces a continuous displacement of the roots of $g(t, -)$! discriminant locus. We want to compute the associated braid.

Applications

- π_1 of the complement of a curve in $\mathbb{CP}^2$ (Marco-Buzunariz & Rodríguez, 2016).
- Action of the monodromy on the homology of a fibration (Pichon-Pharabod, 2024).
- Compute the topology of complex algebraic surfaces.

Approach

Numerically track the roots using certified homotopy continuation.

! We only get an isolating box for each root at all times.

Geometric braids

n points moving without colliding in $\mathbb{C}$
up to homotopy

**Combinatorial braids**

free group words in the letters $\sigma_1, \dots, \sigma_{n-1}$
up to explicit relations



Geometric braids

n points moving without colliding in $\mathbb{C}$
up to homotopy

 $\leftrightarrow$ **Combinatorial braids**

free group words in the letters $\sigma_1, \dots, \sigma_{n-1}$
up to explicit relations



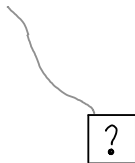
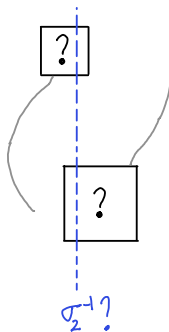
Geometric braids

n points moving without colliding in $\mathbb{C}$
up to homotopy

 $\leftrightarrow$

Combinatorial braids

free group words in the letters $\sigma_1, \dots, \sigma_{n-1}$
up to explicit relations

 $\mathbb{C}$


σ_2^{-1}

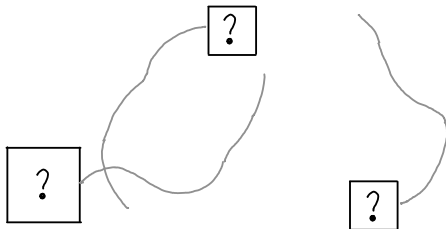
Geometric braids

n points moving without colliding in $\mathbb{C}$
up to homotopy

 $\leftrightarrow$

Combinatorial braids

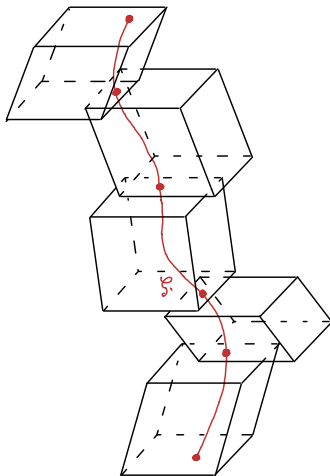
free group words in the letters $\sigma_1, \dots, \sigma_{n-1}$
up to explicit relations



✓ σ_2^{-1}

SIROCCO (Marco-Buzunariz & Rodríguez, 2016)

- Tubular neighborhoods are piecewise **linear**.
- For each strand ζ_i , computes a piecewise linear path in the tube.
- “Intuitive” (! non generic cases) algorithm on the braid with piecewise linear strands.

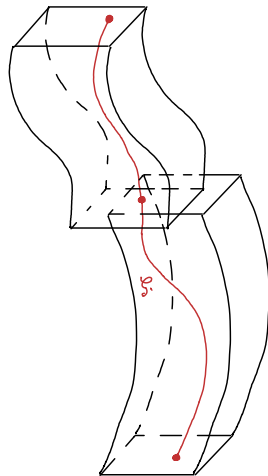


SIROCCO (Marco-Buzunariz & Rodríguez, 2016)

- Tubular neighborhoods are piecewise **linear**.
- For each strand ζ_i , computes a piecewise linear path in the tube.
- “Intuitive” (! non generic cases) algorithm on the braid with piecewise linear strands.

Algpath

- Tubular neighborhoods are piecewise **cubic**.
- ⊕ Faster than SIROCCO.
- ! Finding a piecewise linear path in the tube requires additional work.

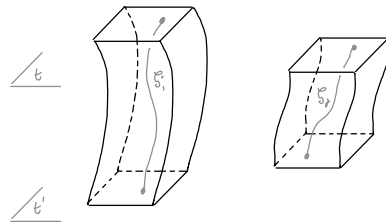


Strand separation interface

We assume a function $\text{Sep}(i, j, t)$ that returns $t' \in (t, 1]$ and a symbol in $\star \in \{\rightarrow, \leftarrow, \rightarrow, \leftarrow\}$, such that for all $s \in [t, t']$:

- $\text{Re}(\zeta_i(s)) < \text{Re}(\zeta_j(s))$ if $\star = \rightarrow$;
- $\text{Re}(\zeta_i(s)) > \text{Re}(\zeta_j(s))$ if $\star = \leftarrow$;
- $\text{Im}(\zeta_i(s)) < \text{Im}(\zeta_j(s))$ if $\star = \rightarrow$;
- $\text{Im}(\zeta_i(s)) > \text{Im}(\zeta_j(s))$ if $\star = \leftarrow$.

Sep characterizes the braid!



$$\text{Sep}(i, j, t) = (t', \rightarrow)$$

Algorithm

Input. Braid ζ given by a Sep predicate.

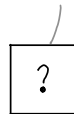
Output. The associated combinatorial braid.



Algorithm

Input. Braid ζ given by a Sep predicate.

Output. The associated combinatorial braid.



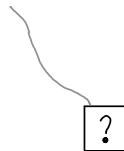
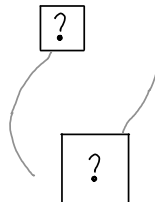
Algorithm

Input. Braid ζ given by a Sep predicate.

Output. The associated combinatorial braid.



Ⓢ



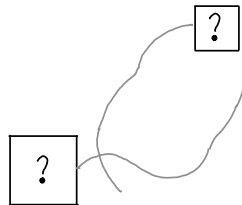
Algorithm

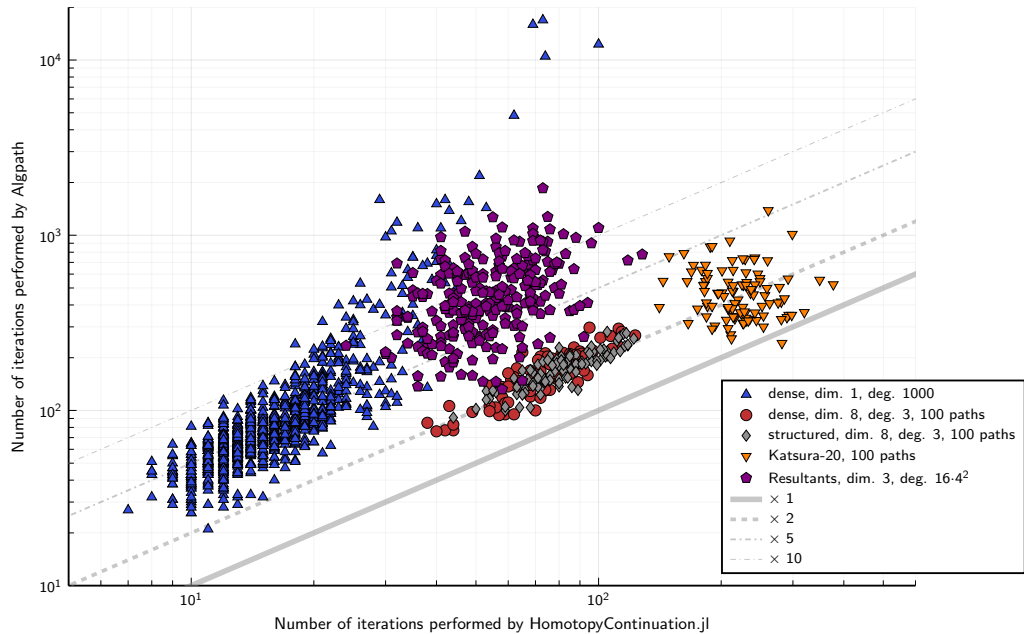
Input. Braid ζ given by a Sep predicate.

Output. The associated combinatorial braid.



Ⓒ





Taylor models

Definition

- A real interval S containing zero,
- a polynomial $P(\eta) = A_0 + A_1\eta + \cdots + A_{d+1}\eta^{d+1}$ where A_i is a (complex) interval.

d is the order of the Taylor model.

Taylor models

Definition

- A real interval S containing zero,
- a polynomial $P(\eta) = A_0 + A_1\eta + \cdots + A_{d+1}\eta^{d+1}$ where A_i is a (complex) interval.

d is the order of the Taylor model.

Definition

A Taylor model (S, P) of degree d encloses a function $\mathcal{X} : \mathbb{R} \rightarrow \mathbb{C}$ if

$$\forall s \in S, \forall i \in 0, \dots, d+1, \exists a_i \in A_i \text{ s.t. } \mathcal{X}(s) = a_0 + a_1s + \cdots + a_{d+1}s^{d+1}.$$

Taylor models

Definition

- A real interval S containing zero,
- a polynomial $P(\eta) = A_0 + A_1\eta + \cdots + A_{d+1}\eta^{d+1}$ where A_i is a (complex) interval.

d is the order of the Taylor model.

Definition

A Taylor model (S, P) of degree d encloses a function $\mathcal{X} : \mathbb{R} \rightarrow \mathbb{C}$ if

$$\forall s \in S, \forall i \in 0, \dots, d+1, \exists a_i \in A_i \text{ s.t. } \mathcal{X}(s) = a_0 + a_1s + \cdots + a_{d+1}s^{d+1}.$$

Taylor models

If $J \subseteq S$, we have $\mathcal{X}(J) \subseteq P(J)$ (where $P(J)$ is computed using interval arithmetic).

Taylor models

Reduction

Let (S, P) be a Taylor model of order d .

Goal : reduce it order to $d - 1$, s.t. if (S, P) encloses a function, so does its reduction.

Solution : replace $A_d \eta^d + A_{d+1} \eta^{d+1}$ by $[A_d \boxplus (A_{d+1} \boxtimes S)] \eta^d$.

Taylor models

Reduction

Let (S, P) be a Taylor model of order d .

Goal : reduce it order to $d - 1$, s.t. if (S, P) encloses a function, so does its reduction.

Solution : replace $A_d \eta^d + A_{d+1} \eta^{d+1}$ by $[A_d \boxplus (A_{d+1} \boxtimes S)] \eta^d$.

Let (S, P) and (S, Q) be Taylor models of order d .

Sum

Component-wise sum of P and Q using $\boxplus$. Compatible with sums of enclosed functions.

Product

Usual product formula with P and Q , gives a Taylor model of order $2d + 1$, then reduce it to make it of order d . Compatible with products of enclosed functions.

Taylor models

- (z, B, A) is a $\frac{1}{8}$ -Moore box for F_t ,
- $\mathcal{X} : \mathbb{R} \rightarrow \mathbb{C}^n$ polynomial s.t. $\mathcal{X}(0) = z$.

We want to check that for all $s \in [0, \delta]$, $(\mathcal{X}(s), B, A)$ is a $\frac{7}{8}$ -Moore box for F_{t+s} . i.e.

$$-A \cdot F_{t+s}(\mathcal{X}(s)) + [1 - A \cdot dF_{t+s}(\mathcal{X}(s) + B)]B \subseteq \frac{7}{8}B.$$

Taylor models

- (z, B, A) is a $\frac{1}{8}$ -Moore box for F_t ,
- $\mathcal{X} : \mathbb{R} \rightarrow \mathbb{C}^n$ polynomial s.t. $\mathcal{X}(0) = z$.

We want to check that for all $s \in [0, \delta]$, $(\mathcal{X}(s), B, A)$ is a $\frac{7}{8}$ -Moore box for F_{t+s} . i.e.

$$-A \cdot F_{t+s}(\mathcal{X}(s)) + [1 - A \cdot dF_{t+s}(\mathcal{X}(s) + B)]B \subseteq \frac{7}{8}B.$$

Taylor models

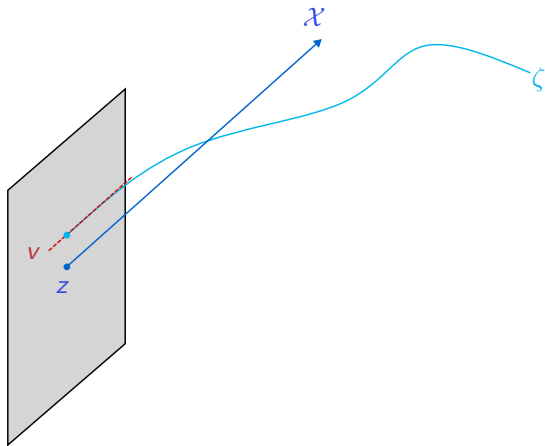
- Compute an order d Taylor model $\mathcal{K}$ on $[0, \delta]$ enclosing

$$s \mapsto -A \cdot F_{t+s}(\mathcal{X}(s)) + [1 - A \cdot dF_{t+s}(\mathcal{X}(s) + B)]B.$$

This is just Taylor model arithmetic !

- Check that $\mathcal{K}([0, \delta]) \subseteq \frac{7}{8}B$ (interval arithmetic).

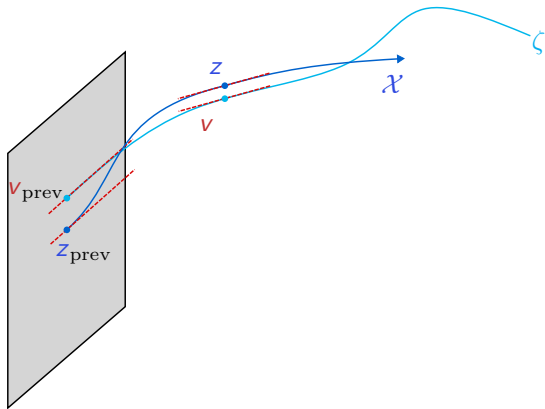
Predictor



Tangent predictor

Idea: $-A \cdot \frac{\partial F}{\partial t}(t, z)$ is a good approximation of $\zeta'(t)$. Use it to do a order 1 correction.

Predictor



Tangent predictor

Idea: $-A \cdot \frac{\partial F}{\partial t}(t, z)$ is a good approximation of $\zeta'(t)$. Use it to do a order 1 correction.

Hermite predictor

Idea: use previous point z_{prev} and previous tangent vector v_{prev} , z and v to do a Hermite cubic spline approximation.

Number field coefficients

LMFDB data representation

- coefficient field $\mathbb{Q}(\nu)$, given by its minimal polynomial m_ν
- Equations for X and ϕ : coefficients in $\mathbb{Q}(\nu)$ (polynomials in ν)
- Each embedding of $\mathbb{Q}(\nu)$ in $\mathbb{C}$ (a root of m_ν) yields a Belyi map
- For each embedding, the LMFDB provides the monodromy of the associated Belyi map

Entry 5T3-4.1_4.1_2.2.1-a: $m_\nu = T^2 + 1$, $X = \mathbb{P}^1$, $\phi : \frac{1}{4107} \frac{(136\nu+4623)x^5+(15096\nu+513153)x^4}{x^5+(-40\nu+55)x^4+(-13236\nu-12048)x^3+\dots}$

Number field coefficients

LMFDB data representation

- coefficient field $\mathbb{Q}(\nu)$, given by its minimal polynomial m_ν
- Equations for X and ϕ : coefficients in $\mathbb{Q}(\nu)$ (polynomials in ν)
- Each embedding of $\mathbb{Q}(\nu)$ in $\mathbb{C}$ (a root of m_ν) yields a Belyi map
- For each embedding, the LMFDB provides the monodromy of the associated Belyi map

Entry 5T3-4.1_4.1_2.2.1-a: $m_\nu = T^2 + 1$, $X = \mathbb{P}^1$, $\phi : \frac{1}{4107} \frac{(136\nu+4623)x^5+(15096\nu+513153)x^4}{x^5+(-40\nu+55)x^4+(-13236\nu-12048)x^3+\dots}$

Problem

The parametrized system F we build to compute the monodromy of a Belyi map may have number field coefficients. How do we handle that in Algpah?

Number field coefficients

Assume a parametrized system F that has coefficients in $\mathbb{Q}(\nu) \leftarrow$ given by m_ν .

Assume an embedding of $\mathbb{Q}(\nu)$ into $\mathbb{C}$, or equivalently α a root of m_ν .

Number field coefficients

Assume a parametrized system F that has coefficients in $\mathbb{Q}(\nu) \leftarrow$ given by m_ν .
Assume an embedding of $\mathbb{Q}(\nu)$ into $\mathbb{C}$, or equivalently α a root of m_ν .

Solution 1

System: $\{F_t(x) = 0$ Variables: x Starting solution: ζ_0

Number field coefficients

Assume a parametrized system F that has coefficients in $\mathbb{Q}(\nu) \leftarrow$ given by m_ν .

Assume an embedding of $\mathbb{Q}(\nu)$ into $\mathbb{C}$, or equivalently α a root of m_ν .

Solution 1

$$\text{System: } \begin{cases} F_t(x) = 0 \\ m_\nu = 0 \end{cases} \quad \text{Variables: } x, \nu \quad \text{Starting solution: } \zeta_0, \alpha$$

Number field coefficients

Assume a parametrized system F that has coefficients in $\mathbb{Q}(\nu) \leftarrow$ given by m_ν .
Assume an embedding of $\mathbb{Q}(\nu)$ into $\mathbb{C}$, or equivalently α a root of m_ν .

Solution 1

$$\text{System: } \begin{cases} F_t(x) = 0 \\ m_\nu = 0 \end{cases} \quad \text{Variables: } x, \nu \quad \text{Starting solution: } \zeta_0, \alpha$$

 Termination

 Slows down computations  rectangular isolating regions

Number field coefficients

Assume a parametrized system F that has coefficients in $\mathbb{Q}(\nu) \leftarrow$ given by m_ν .
Assume an embedding of $\mathbb{Q}(\nu)$ into $\mathbb{C}$, or equivalently α a root of m_ν .

Solution 1

$$\text{System: } \begin{cases} F_t(x) = 0 \\ m_\nu = 0 \end{cases} \quad \text{Variables: } x, \nu \quad \text{Starting solution: } \zeta_0, \alpha$$

+ Termination

- Slows down computations ⚙ rectangular isolating regions

Solution 2 (what we use in practice)

Replace ν by an interval enclosing $\alpha \rightsquigarrow F$ has interval coefficients

- The software may stall if the interval is too wide.

+ Faster compared to Solution 1

Arrangements

$$OC_n = \{(x_1, \dots, x_n) \in \mathbb{C}^n : \forall i \neq j, x_i \neq x_j\}.$$

Arrangement

Pair $(\prec_R, \prec_I)$ of partial orders on $\{1, \dots, n\}$, such that all $i \neq j$ are comparable by $\prec_R$ or $\prec_I$.

Arrangement cell

Set of points $(x_1, \dots, x_n) \in OC_n$ such that

- for all $i \prec_R j$, $\operatorname{Re}(x_i) < \operatorname{Re}(x_j)$,
- for all $i \prec_I j$, $\operatorname{Im}(x_i) < \operatorname{Im}(x_j)$,

In practice, we represent a partial order with a DAG.

Arrangements

$$OC_n = \{(x_1, \dots, x_n) \in \mathbb{C}^n : \forall i \neq j, x_i \neq x_j\}.$$

Arrangement

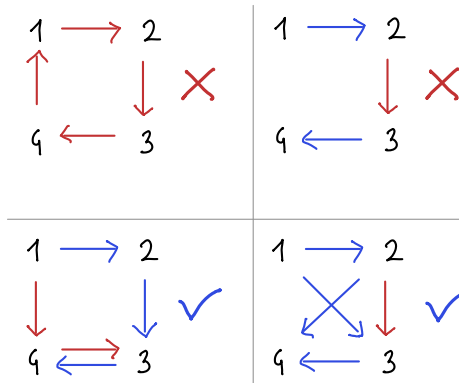
Pair $(\prec_R, \prec_I)$ of partial orders on $\{1, \dots, n\}$, such that all $i \neq j$ are comparable by $\prec_R$ or $\prec_I$.

Arrangement cell

Set of points $(x_1, \dots, x_n) \in OC_n$ such that

- for all $i \prec_R j$, $\text{Re}(x_i) < \text{Re}(x_j)$,
- for all $i \prec_I j$, $\text{Im}(x_i) < \text{Im}(x_j)$,

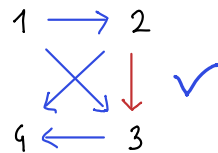
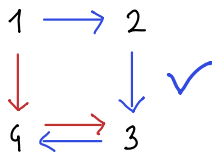
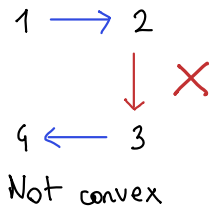
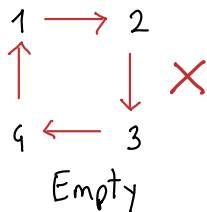
In practice, we represent a partial order with a DAG.



Arrangements

Property

Arrangement cells are non-empty and convex subsets of OC_n .

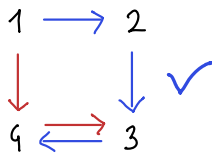
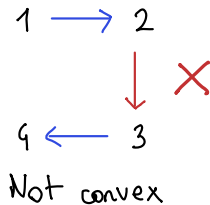
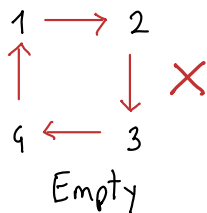


Non empty and convex

Arrangements

Property

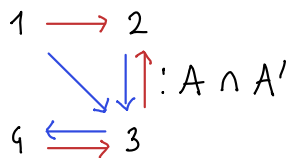
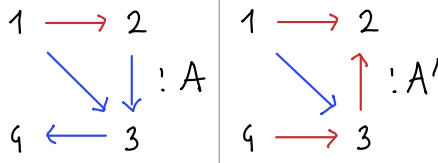
Arrangement cells are non-empty and convex subsets of OC_n .



Non empty and convex

Property

Non-empty intersection of arrangement cells is an arrangement cell.

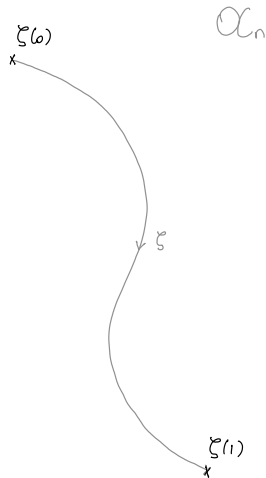


$$(2i, 1+3i, 2+i, 4) \in A \cap A'$$

Step 1: compute a sequence of arrangements

Path to cells

Input: ζ , given by its Sep function.



Step 1: compute a sequence of arrangements

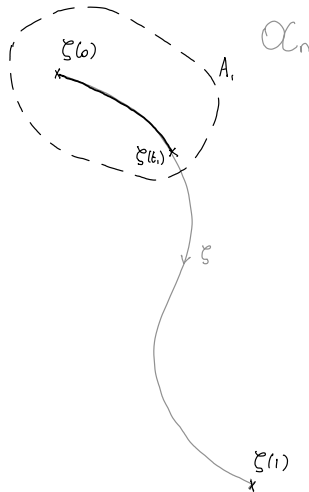
Path to cells

Input: ζ , given by its Sep function.

Output: a sequence of arrangement cells $A_1, \dots, A_r$ such that there exists $0 = t_0 < \dots < t_r = 1$ and for any $s \in [t_{i-1}, t_i]$, $\zeta(s) \in A_i$.

Idea:

- Start with an arrangement cell A containing $\zeta(0)$.
- Associate to each edge a time of validity.
- When a relation expires, update it using Sep and repair convexity.
- Repeat.



Step 1: compute a sequence of arrangements

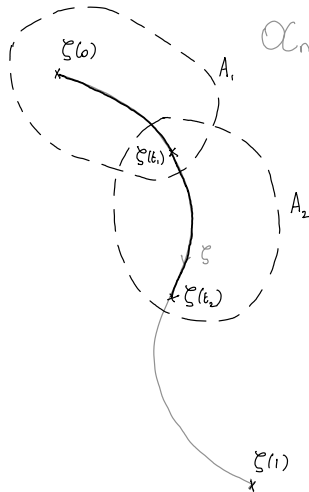
Path to cells

Input: ζ , given by its Sep function.

Output: a sequence of arrangement cells $A_1, \dots, A_r$ such that there exists $0 = t_0 < \dots < t_r = 1$ and for any $s \in [t_{i-1}, t_i]$, $\zeta(s) \in A_i$.

Idea:

- Start with an arrangement cell A containing $\zeta(0)$.
- Associate to each edge a time of validity.
- When a relation expires, update it using Sep and repair convexity.
- Repeat.



Step 1: compute a sequence of arrangements

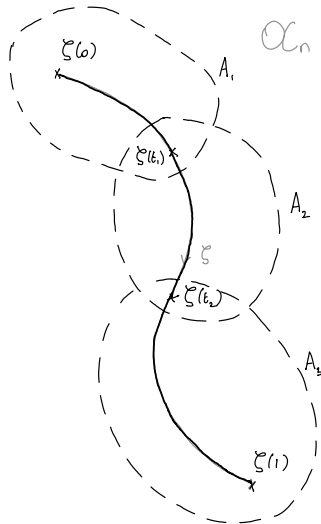
Path to cells

Input: ζ , given by its Sep function.

Output: a sequence of arrangement cells $A_1, \dots, A_r$ such that there exists $0 = t_0 < \dots < t_r = 1$ and for any $s \in [t_{i-1}, t_i]$, $\zeta(s) \in A_i$.

Idea:

- Start with an arrangement cell A containing $\zeta(0)$.
- Associate to each edge a time of validity.
- When a relation expires, update it using Sep and repair convexity.
- Repeat.



Step 2: linearize ζ

Permutation points

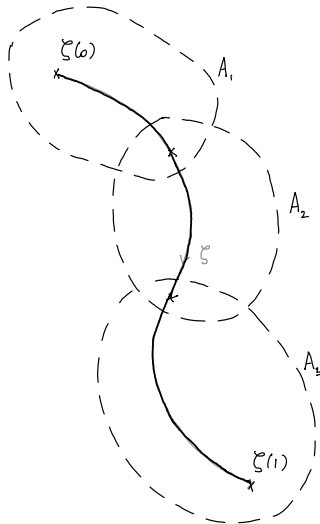
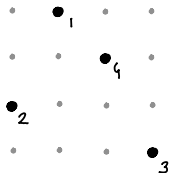
Let $\pi, \varphi \in \mathfrak{S}_n$. We define

$$p_{\pi, \varphi} = (\pi(1) + \mathbf{i}\varphi(1), \dots, \pi(n) + \mathbf{i}\varphi(n)) \in OC_n.$$

$$\pi = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 & 1 & 4 & 3 \end{pmatrix}$$

$$e = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 4 & 2 & 1 & 3 \end{pmatrix}$$

$p_{\pi, e} :$



Step 2: linearize ζ

Permutation points

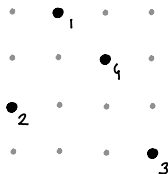
Let $\pi, \varphi \in \mathfrak{S}_n$. We define

$$p_{\pi, \varphi} = (\pi(1) + \mathbf{i}\varphi(1), \dots, \pi(n) + \mathbf{i}\varphi(n)) \in OC_n.$$

$$\pi = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 & 1 & 4 & 3 \end{pmatrix}$$

$$\varphi = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 4 & 2 & 1 & 3 \end{pmatrix}$$

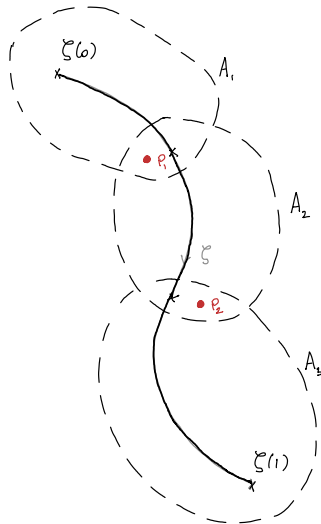
$p_{\pi, \varphi} e :$



Linearization of ζ

For each A_i, A_{i+1} , we compute π, φ such that

$p_i = p_{\pi, \varphi}$ lies in the intersection $A_i \cap A_{i+1}$ (Hint: total order extending $\prec_R$ and $\prec_I$).



Step 2: linearize ζ

Permutation points

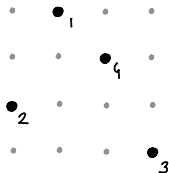
Let $\pi, \varphi \in \mathfrak{S}_n$. We define

$$p_{\pi, \varphi} = (\pi(1) + \mathbf{i}\varphi(1), \dots, \pi(n) + \mathbf{i}\varphi(n)) \in OC_n.$$

$$\pi = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 & 1 & 4 & 3 \end{pmatrix}$$

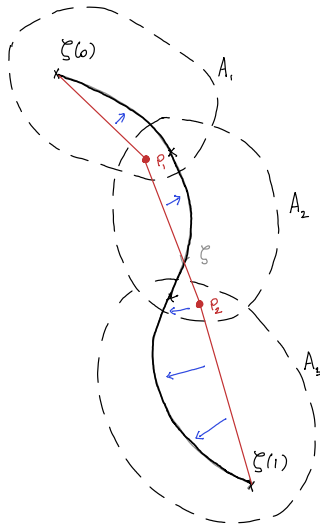
$$\varphi = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 4 & 2 & 1 & 3 \end{pmatrix}$$

$p_{\pi, \varphi} e :$



Linearization of ζ

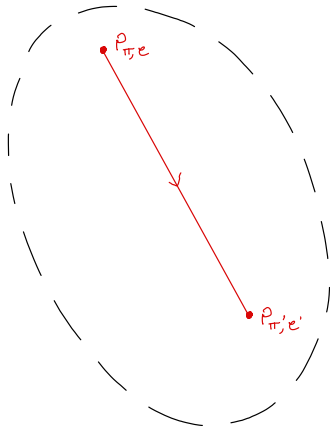
For each A_i, A_{i+1} , we compute π, φ such that $p_i = p_{\pi, \varphi}$ lies in the intersection $A_i \cap A_{i+1}$ (Hint: total order extending $\prec_R$ and $\prec_I$). The linear interpolation of the p_i is homotopic to ζ . Why? cells are convex!



Decomposition of the linearization

Reduction

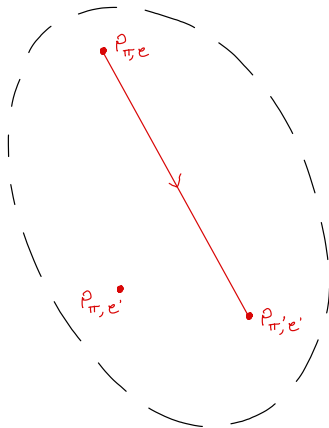
- Computing the braid associated to the whole linearization or to each piece and concatenating the results is equivalent.



Decomposition of the linearization

Reduction

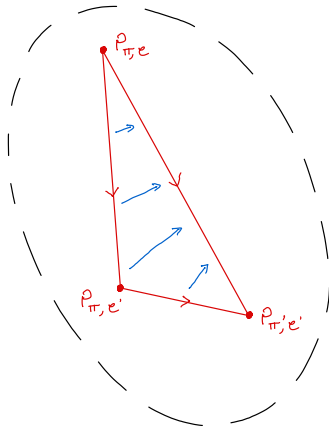
- Computing the braid associated to the whole linearization or to each piece and concatenating the results is equivalent.
- Assume $p_{\pi,\varphi}$ and $p_{\pi',\varphi'}$ both lie in an arrangement $(\prec_R, \prec_I)$. It means that π, π' extend $\prec_R$ and φ, φ' extend $\prec_I$. **So $p_{\pi,\varphi'}$ also lies in c !**



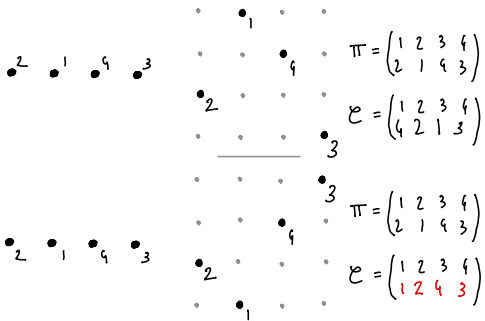
Decomposition of the linearization

Reduction

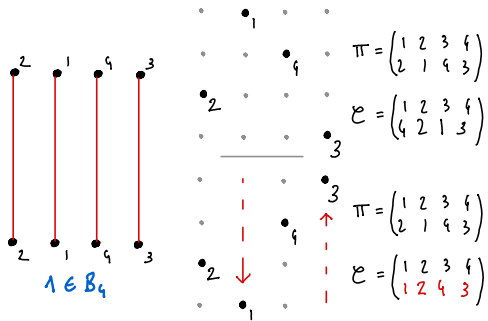
- Computing the braid associated to the whole linearization or to each piece and concatenating the results is equivalent.
- Assume $p_{\pi,\varphi}$ and $p_{\pi',\varphi'}$ both lie in an arrangement $(\prec_R, \prec_I)$. It means that π, π' extend $\prec_R$ and φ, φ' extend $\prec_I$. **So $p_{\pi,\varphi'}$ also lies in c !**
- We compute the braid of $p_{\pi,\varphi} \rightarrow p_{\pi,\varphi'}$ (trivial) then the braid of $p_{\pi,\varphi'} \rightarrow p_{\pi',\varphi'}$ (formula)



Decomposition of the linearization

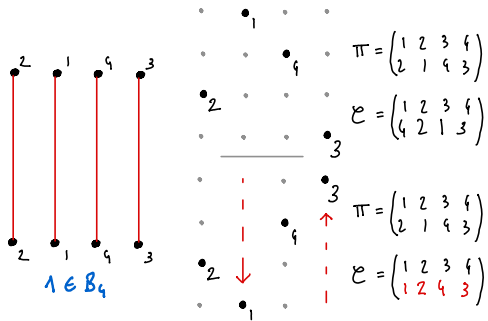


Decomposition of the linearization

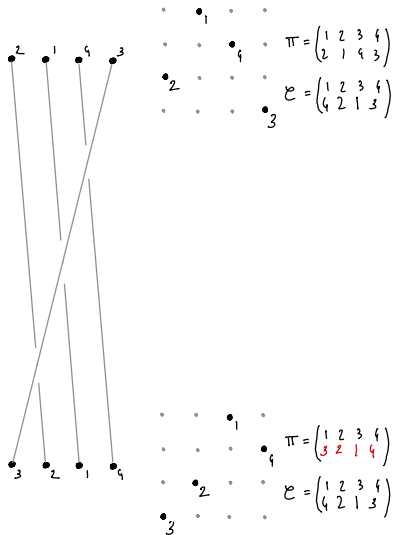


$p_{\pi, \varphi} \rightarrow p_{\pi, \varphi'}: \text{trivial braid.}$

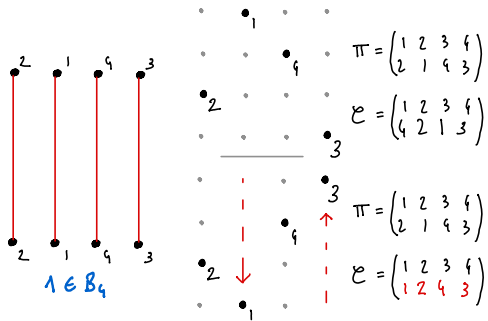
Decomposition of the linearization



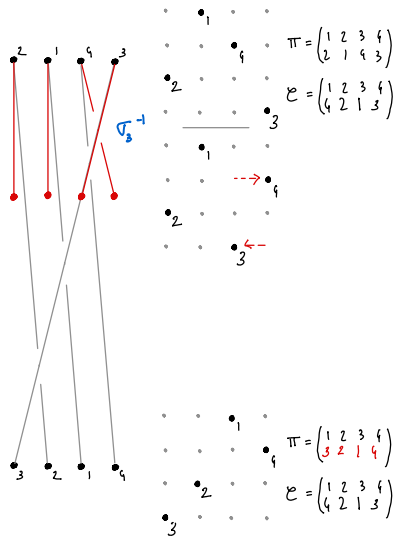
$p_{\pi, \varphi} \rightarrow p_{\pi, \varphi'}: \text{trivial braid.}$



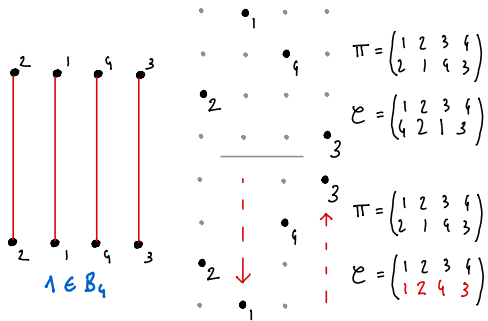
Decomposition of the linearization



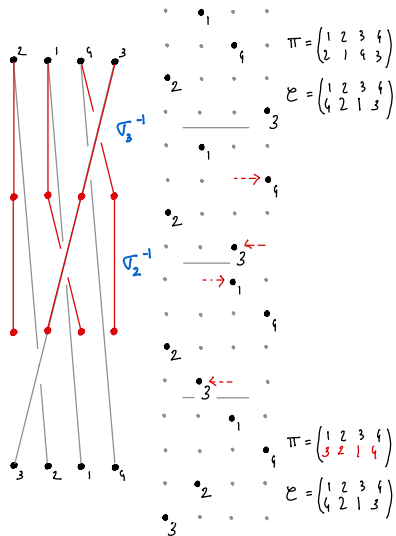
$p_{\pi, \varphi} \rightarrow p_{\pi, \varphi'}: \text{trivial braid.}$



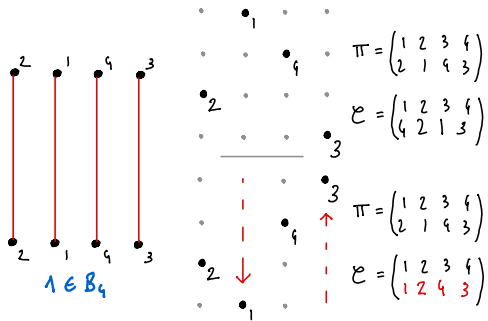
Decomposition of the linearization



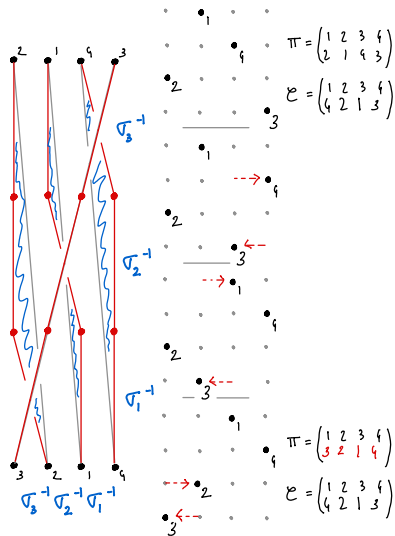
$p_{\pi, \varphi} \rightarrow p_{\pi, \varphi'}: \text{trivial braid.}$



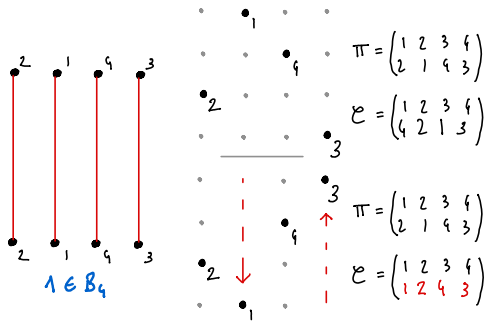
Decomposition of the linearization



$p_{\pi, \varphi} \rightarrow p_{\pi, \varphi'}: \text{trivial braid.}$

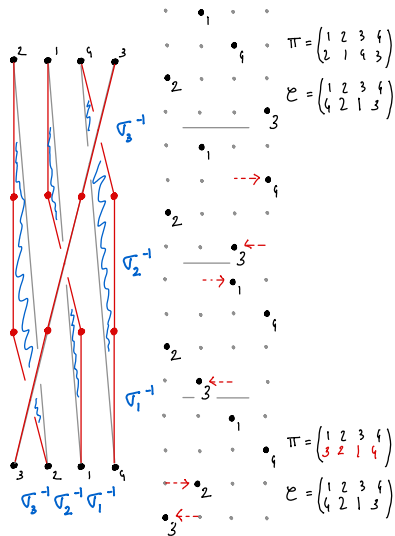


Decomposition of the linearization



$p_{\pi, \varphi} \rightarrow p_{\pi, \varphi'}$: trivial braid.

$p_{\pi, \varphi'} \rightarrow p_{\pi', \varphi'}$: Decompose $\pi' \pi^{-1} = s_{i_1} \cdots s_{i_r}$ in elementary transpositions. Output $\sigma_{i_1}^{\varepsilon_1} \cdots \sigma_{i_r}^{\varepsilon_r}$ with $\varepsilon_1, \dots, \varepsilon_r \in \{\pm 1\}$ computed using φ' .





Barth, D., & Wenz, A. (2021). Computation of Belyi maps with prescribed ramification and applications in Galois theory. *Journal of Algebra*, 569, 616–642.



Bartholdi, L., Buff, X., Graf Von Bothmer, H.-C., & Kröker, J. (2015). Algorithmic Construction of Hurwitz Maps. *Experimental Mathematics*, 24(1), 76–92.



Bates, D. J., Sommese, A. J., Hauenstein, J. D., & Wampler, C. W. (2013). *Numerically Solving Polynomial Systems with Bertini*. Society for Industrial; Applied Mathematics.



Beltrán, C. (2011). A continuation method to solve polynomial systems and its complexity. *Numer. Math.*, 117(1), 89–113.



Beltrán, C., & Leykin, A. (2012). Certified Numerical Homotopy Tracking. *Experimental Mathematics*, 21(1), 69–83.



Beltrán, C., & Leykin, A. (2013). Robust Certified Numerical Homotopy Tracking. *Found Comput Math*, 13(2), 253–295.



Belyi, G. V. (1980). On Galois Extensions Of a Maximal Cyclotomic Field. *Math. USSR Izv.*, 14(2), 247.



Belyi map verification using certified path tracking. (2026). To appear in ICMS proceedings.



Breiding, P., Rose, K., & Timme, S. (2023). Certifying Zeros of Polynomial Systems Using Interval Arithmetic. *ACM Trans. Math. Softw.*, 49(1), 11:1–11:14.



Breiding, P., & Timme, S. (2018). HomotopyContinuation.jl: A Package for Homotopy Continuation in Julia. In J. H. Davenport, M. Kauers, G. Labahn, & J. Urban (Eds.), *Mathematical Software – ICMS 2018* (pp. 458–465). Springer International Publishing.



Bruin, N., Sijssling, J., & Zotine, A. (2019). Numerical computation of endomorphism rings of Jacobians. *Open Book Series*, 2(1), 155–171.



Bürgisser, P., & Cucker, F. (2011). On a problem posed by Steve Smale. *Ann. Math.*, 174(3), 1785–1836.



Certified Algebraic Path Tracking with Algpath. (2026). *ACM Commun. Comput. Algebra*, 59(3), 53–56.



Chen, T., Lee, T.-L., & Li, T.-Y. (2014). Hom4PS-3: A Parallel Numerical Solver for Systems of Polynomial Equations Based on Polyhedral Homotopy Continuation Methods. In H. Hong & C. Yap (Eds.), *Math. Softw. – ICMS 2014* (pp. 183–190). Springer.



Computing braids from approximate data. (2026). arXiv: 2601.23073 [cs].



Deconinck, B., & van Hoeij, M. (2001). Computing Riemann matrices of algebraic curves. *Physica D: Nonlinear Phenomena*, 152–153, 28–46.



Dedieu, J.-P., Malajovich, G., & Shub, M. (2013). Adaptive step-size selection for homotopy methods to solve polynomial equations. *IMA J. Numer. Anal.*, 33(1), 1–29.



Drexler, F. J. (1977). Eine Methode zur Berechnung sämtlicher Lösungen von Polynomgleichungssystemen.. *Numer. Math.*, 29, 45–58.



Duff, T., & Lee, K. (2024). Certified homotopy tracking using the Krawczyk method. *Proc. ISSAC 2024*, 274–282.



Duff, T., & Lee, K. (2026). Certifying Galois/monodromy Actions via Homotopy Graphs.



Elkies, N. (2013). The complex polynomials $P(x)$ with $\text{Gal}(P(x) - t) \cong M_{23}$. *Open Book Series*, 1(1), 359–367.



Garcia, C. B., & Zangwill, W. I. (1979). Finding all solutions to polynomial systems and other systems of equations. *Math. Program.*, 16(1), 159–176.



Gunji, T., Kim, S., Kojima, M., Takeda, A., Fujisawa, K., & Mizutani, T. (2004). PHoM – a Polyhedral Homotopy Continuation Method for Polynomial Systems. *Computing*, 73(1).



Hauenstein, J. D., Haywood, I., & Liddell, A. C. (2014). An a posteriori certification algorithm for Newton homotopies. *Proceedings of the 39th International Symposium on Symbolic and Algebraic Computation*, 248–255.



Hauenstein, J. D., & Liddell, A. C. (2016). Certified predictor–corrector tracking for Newton homotopies. *Journal of Symbolic Computation*, 74, 239–254.



Hauenstein, J. D., & Sottile, F. (2012). Algorithm 921: alphaCertified: Certifying Solutions to Polynomial Systems. *ACM Trans. Math. Softw. TOMS*, 38(4), 28:1–28:20.



Johansson, F. (2017). Arb: Efficient Arbitrary-Precision Midpoint-Radius Interval Arithmetic. *IEEE Trans. Comput.*, 66(8), 1281–1292.



Kearfott, R. B., & Xing, Z. (1994). An Interval Step Control for Continuation Methods. *SIAM J. Numer. Anal.*, 31(3), 892–914.



Klug, M., Musty, M., Schiavone, S., & Voight, J. (2014). Numerical calculation of three-point branched covers of the projective line. *LMS J. Comput. Math.*, 17(1), 379–430.



Kranich, S. (2016). An epsilon-delta bound for plane algebraic curves and its use for certified homotopy continuation of systems of plane algebraic curves.



Krawczyk, R. (1969). Newton-Algorithmen zur Bestimmung von Nullstellen mit Fehlerschranken. *Computing*, 4(3), 187–201.



Leykin, A. (2011). Numerical algebraic geometry. *J. Softw. Algebra Geom.*, 3(1), 5–10.



Marco-Buzunariz, M. Á., & Rodríguez, M. (2016). SIROCCO: A Library for Certified Polynomial Root Continuation. In G.-M. Greuel, T. Koch, P. Paule, & A. Sommese (Eds.), *Mathematical Software – ICMS 2016* (pp. 191–197). Springer International Publishing.



Moore, R. E. (1977). A Test for Existence of Solutions to Nonlinear Systems. *SIAM Journal on Numerical Analysis*, 14(4), 611–615.



Pichon-Pharabod, E. (2024). *Periods in algebraic geometry : Computations and application to Feynman integrals* (Theses 2024UPASM034). Université Paris-Saclay.



Roberts, D. P. (2018). Hurwitz–Belyi maps. *Publications mathématiques de Besançon. Algèbre et théorie des nombres*, 25–67.



Rump, S. M. (1983). SOLVING ALGEBRAIC PROBLEMS WITH HIGH ACCURACY. In U. W. Kulisch & W. L. Miranker (Eds.), *A New Approach to Scientific Computation* (pp. 51–120). Academic Press.



Schneps, L. (Ed.). (1994). Dessins d'enfants on the Riemann sphere. In *The Grothendieck Theory of Dessins d'Enfants* (pp. 47–78). Cambridge University Press.



Shub, M., & Smale, S. (1994). Complexity of Bezout's theorem V: Polynomial time. *Theor. Comput. Sci.*, 133(1), 141–164.



Shub, M., & Smale, S. (1993). Complexity of Bezout's Theorem I: Geometric Aspects. *J. Am. Math. Soc.*, 6(2), 459–501.



Sijsling, J., & Voight, J. (2015). On computing Belyi maps. *Publications mathématiques de Besançon. Algèbre et théorie des nombres*, (1), 73–131.



Smale, S. (1986). Newton's Method Estimates from Data at One Point. In R. E. Ewing, K. I. Gross, & C. F. Martin (Eds.), *Merging Discip. New Dir. Pure Appl. Comput. Math.* (pp. 185–196). Springer.



Su, H.-J., McCarthy, J. M., Sosonkina, M., & Watson, L. T. (2006). Algorithm 857: POLSYS_GLP—a parallel general linear product homotopy code for solving polynomial systems of equations. *ACM Trans. Math. Softw.*, 32(4), 561–579.



Telen, S., Van Barel, M., & Verschelde, J. (2020). A Robust Numerical Path Tracking Algorithm for Polynomial Homotopy Continuation. *SIAM J. on Sci. Comput.*, 42(6), A3610–A3637.



Validated Numerics for Algebraic Path Tracking. (2024). *Proc. ISSAC 2024*, 36–45.



van der Hoeven, J. (2015). *Reliable homotopy continuation* (Research Report). LIX, Ecole polytechnique.



Verschelde, J. (1999). Algorithm 795: PHCpack: A general-purpose solver for polynomial systems by homotopy continuation. *ACM Trans. Math. Softw.*, 25(2), 251–276.



Xu, J., Burr, M., & Yap, C. (2018). An Approach for Certifying Homotopy Continuation Paths: Univariate Case. *Proc. ISSAC 2018*, 399–406.